# 1024-HDC Research — End-to-End Knowledge Guide

**Purpose:** Everything you should understand to explain this project clearly to your professor — from HDC fundamentals through RTL, verification, bus interfaces, research contributions, and remaining work.

**Project:** Streaming 1024-bit Hyperdimensional Computing classifier on Xilinx Zynq for EMG hand-gesture recognition.

**Repository:** `harsha240yeager/1024-HDC`

**Target venue:** DATE 2027 (~September 2026 submission)

---

## Table of contents



---

## 1. One-sentence pitch

> I am building a **streaming 1024-bit Hyperdimensional Computing (HDC) classifier on Zynq** for EMG hand-gesture recognition. The RTL is **bit-exact verified** against a Python golden reference through seven automated co-simulation harnesses. The research contribution is a **three-axis accuracy/energy Pareto study** (dimension × bundle precision × bit-pruning), plus **informed vs random pruning** and **cross-subject mask transfer** — jointly evaluated on real hardware, which prior FPGA-HDC work has not done in this structured way.

**Working title (from advisor one-pager):** *Informed Bit-Position Pruning for Hyperdimensional Computing on FPGA — A discriminability-driven Pareto study with cross-subject transferability on Zynq, for EMG classification.*

---

## 2. HDC fundamentals

### 2.1 What is Hyperdimensional Computing?

HDC represents information as **very wide vectors** (hypervectors). In this project, vectors are **1024-bit binary** arrays (Binary Spatter Code, BSC model). Operations are simple, bit-parallel, and noise-tolerant — a strong match for FPGA edge inference.

| Term | Meaning | In this project |
|---|---|---|
| **Hypervector** | High-dimensional representation | D = 1024 bits, packed as 16 × 64-bit words |
| **Bind** | Combine two symbols | Bitwise **XOR** |
| **Permute** | Encode position/role | Cyclic shift / reorder (3 modes in RTL) |
| **Bundle** | Accumulate evidence | **Majority vote** across many bound vectors |
| **Associative memory (AM)** | Classify by similarity | **Masked Hamming distance** + argmin over class prototypes |
| **Item memory** | Random hypervector lookup tables | ROMs for channel, feature, and value indices |

### 2.2 Why HDC on FPGA (and not only software)?

- **Bind, permute, popcount** are naturally bit-parallel — no DSP blocks needed.
- **Low dynamic energy** when combined with bit-position pruning (mask gates popcount work).
- **Training is lightweight** — no backprop; prototypes built by bundling encoded windows.
- **Edge biosignals** (EMG) need low latency and predictable power — FPGA + bare-metal PS is a clean measurement story.

### 2.3 Why not just use a tiny neural network?

The research plan requires a **tiny int8 MLP baseline** (~5k parameters) as a rebuttal. HDC offers: no gradient training, interpretable hypervectors, and a principled hardware story. The paper compares both.

---

## 3. Application: EMG encoding pipeline

### 3.1 Dataset

**Primary:** UCI EMG Hand-Gesture dataset (Rahimi et al., ICRC 2016 — canonical HDC benchmark).

| Parameter | Value |
|---|---|
| Channels | 4 |
| Features per channel | 5 |
| Quantization levels | 16 (0…15) |
| Tuples per window | 20 (= 4 × 5) |
| Classes | 5 (paper); co-sim core uses N_CLASS = 8 for generality |
| Subjects | 36 (used in cross-subject Twist 2) |

**Secondary (planned):** MIT-BIH ECG arrhythmia (5 AAMI classes).

### 3.2 One window → one class label (inference path)

**Step 1 — Quantize:** Each EMG sample in the window is mapped to a level 0…15.

**Step 2 — Encode (20 pairs):** For each (channel c, feature f):

```
pair = channel_HV[c]  XOR  value_HV[level]  XOR  permute(feature_HV[f], mode=2,
param=f)
```

This is implemented in `encoder_top.sv` using three `item_mem` ROMs (channel / feature / value tables) and the verified bind+permute datapath.

**Step 3 — Bundle:** Majority vote across all 20 pair hypervectors → **query hypervector** (1024 bits). Implemented in `bundle_unit.sv` with counter width `CNT_W` (default 6 bits).

**Step 4 — Classify:** Compare query to each stored **class prototype** using **masked Hamming distance**:

```
distance(class) = popcount( (query XOR prototype) AND mask )
predicted_class = argmin(distance)
```

Implemented in `popcount_am.sv`. The pruning **mask** is a per-bit enable: cleared bits are ignored in the distance sum.

### 3.3 Data flow diagram (draw on whiteboard)

```
EMG window (80-bit level grid: 20 tuples × 4-bit levels)
    |
    ▼
encoder_top ── item_mem ROMs + bind/permute + bundle_unit
    |
    ▼
query hypervector (1024 bits)
    |
    ▼
popcount_am ── masked Hamming to N_CLASS prototypes → argmin
    |
```

```
    ▼
class_idx, class_dist
```

**Rough latency:** encode ≈ N_PAIRS (20) + a few cycles + AM (1 cycle). End-to-end core latency is on the order of ~25 clock cycles at the configured clock.

---

## 4. Training vs inference

| Phase | Where it runs | What happens |
|---|---|---|
| **Training** | Python (offline) or C on PS (bare-metal) | Encode many windows per class; accumulate votes; threshold to binary → **class prototype**; compute **Fisher-informed pruning mask** |
| **Configuration** | PS via AXI4-Lite | Load prototypes and mask into PL staging registers / AM |
| **Inference** | PL core (optionally fed by AXI-DMA stream) | One window in → one class out |

**Important distinction:** Learning happens in software by **bundling** encoded queries into prototypes. The FPGA executes **inference only** (encode window + nearest-prototype search). Pruning masks are computed offline in Python ( `make_pruning_masks` in `hdc_ref.py` ) and loaded at run time.

---

## 5. Hardware architecture

### 5.1 Module stack (bottom to top)

| Module | Role | Verification harness |
|---|---|---|
| `permute_stage.sv` + `xor_permute_top.sv` | Bind (XOR) + permute primitives | `sim/run_cosim.do` |
| `bundle_unit.sv` | Majority bundler | `sim/run_bundle_cosim.do` — 500/500 PASS |
| `popcount_am.sv` | Masked Hamming + argmin AM | `sim/run_am_cosim.do` — 500/500 PASS |
| `item_mem.sv` + `encoder_top.sv` | EMG window encoder | `sim/run_encoder_cosim.do` — 500/500 PASS |
| `hdc_core_top.sv` | Encoder → AM end-to-end | `sim/run_core_cosim.do` — 500/500 PASS |
| `hdc_core_axi_lite.sv` | Register-mapped PS interface | `sim/run_core_axi_cosim.do` — 200/200 PASS |

| Module | Role | Verification harness |
|---|---|---|
| `hdc_stream_wrapper.sv` | DMA streaming interface | `sim/run_stream_cosim.do` — 200/200 PASS |

All modules are **parameterised on D** (hypervector dimension) for the Hook A Pareto sweep.

### 5.2 `hdc_core_top.sv` — the inference core

Composes `encoder_top` and `popcount_am`:

- **Configuration ports:** `proto_we` / `proto_idx` / `proto_vec` load class prototypes; `mask_we` / `mask_vec` load pruning mask (defaults to all-ones = unmasked after reset).
- **Inference:** Pulse `start` with `levels_flat` (80-bit packed grid); `out_valid` pulses with `class_idx` and `class_dist`.
- **Matches Python:** `HDCEngine.encode_emg_window()` then `HDCEngine.classify()`.

### 5.3 `hdc_stream_wrapper.sv` — the streaming path

- **Input (S_AXIS):** One window = **3 beats** (80 bits in 32-bit TDATA, little-endian); `TLAST` on final beat.
- **Output (M_AXIS): 1 beat** per window: `(class_idx << 16) | class_dist`, `TLAST = 1`.
- **Back-pressure:** `s_axis_tready` drops while core is busy; `m_axis_tvalid` holds until consumed.
- **Configuration:** Prototype/mask write ports passed through to core (from AXI-Lite staging in full Zynq design).

### 5.4 Target platform

- **SoC:** Xilinx Zynq-7020 class (ZedBoard / PYNQ-Z2 per research plan).
- **PS–PL coupling:** AXI4-Lite for control + AXI-DMA for stream (block design — not yet complete).
- **Recommended software:** Bare-metal C first (cleanest energy measurements); PetaLinux optional for demo polish.

---

## 6. Bus interfaces: AXI4-Lite vs AXI4-Stream

| Aspect | AXI4-Lite (`hdc_core_axi_lite.sv`) | AXI4-Stream (`hdc_stream_wrapper.sv`) |
|---|---|---|
| **Purpose** | Control plane: config, staging, debug | Data plane: continuous window streaming |
| **Data movement** | Register reads/writes per window | DMA-fed beat stream |
| **Throughput** | Lower — register overhead per inference | Higher — natural for real-time EMG |
| **Use in paper** | Baseline path; prototype/mask loading | Main deployment path |
| **Co-sim status** | 200/200 PASS | 200/200 PASS (+ random gaps + back-pressure) |

**Key sentence for the professor:** *AXI-Lite is how the PS configures and debugs the core; AXI-Stream is how windows flow at inference rate through DMA.*

Register map (AXI-Lite): CTRL, STATUS, PROTO_IDX, RESULT, LEVELS0…2 (staging), prototype/mask buffer writes. See `docs/HDC_Core_AXI_Lite_and_Cosim_Flow.pdf`.

Protocol primers: `docs/AXI4_Lite_Protocol_Study.pdf`, `docs/AXI4_Stream_Protocol_Study.pdf`.

---

# 7. Verification methodology

## 7.1 Golden reference principle

**Python (`python_ref/hdc_ref.py`) is the single source of truth.** RTL must match bit-for-bit:

- Same seed → same item memories
- Same bind, permute modes, bundle tie-break, masked popcount
- Co-sim testbenches read hex vectors generated by `generate_vectors.py` and compare cycle-by-cycle or at result boundaries

## 7.2 Co-simulation harnesses (all PASS)

| Harness | Cases | What it proves |
|---|---|---|
| `run_cosim.do` | Directed + random | XOR bind + permute |
| `run_bundle_cosim.do` | 500 | Majority bundle |
| `run_am_cosim.do` | 500 | Masked Hamming AM |
| `run_encoder_cosim.do` | 500 | Full window encode |
| `run_core_cosim.do` | 500 | End-to-end encode → classify |
| `run_core_axi_cosim.do` | 200 | AXI-Lite programming sequence |
| `run_stream_cosim.do` | 200 | Streaming + back-pressure + random valid gaps |

Debug variant: `run_stream_cosim_debug.do` (+DEBUG +TRACE=3 +WAVE) for learning the streaming FSM.

## 7.3 What verification does NOT yet cover

- Post-route timing (f_max) and utilisation (LUT/FF/BRAM)
- On-board functional smoke test (VDI access available)
- Full dataset accuracy replay on silicon (±0.5% vs Python)
- Energy measurements (requires bench setup)

---

# 8. Software baseline (what is already proven)

Before hardware experiments, the **algorithm** was locked in software:

| Milestone | Result |
|---|---|
| Stage A — literal MAP parity vs Rahimi 2016 | Spatial **90.36%** (paper 90.8%); spatiotemporal **96.04%** (paper 97.8%) |
| Stage B — RTL-matched binary BSC model | D-sweep completed |
| **Frozen baseline (protocol P-may2026)** | **90.30% ± 0.13 spatial @ D = 1024** |

Config: `python_ref/config/emg_baseline.json`
Results: `python_ref/results/emg_baseline.json`
Notes: `python_ref/notes/emg_baseline_frozen.pdf`

**Why this matters:** You can tell the professor the **classification algorithm is validated** independently of FPGA bring-up. Hardware work is about **implementation efficiency and novel pruning studies**, not fixing a broken classifier.

---

## 9. Research contributions

Three layered hooks — absent as a joint study from prior FPGA-HDC work:

### 9.1 Hook A — Three-axis Pareto (headline experiment)

Joint accuracy / energy / area trade-off across:

| Axis | Values | How varied |
|---|---|---|
| **Dimension D** | 256, 512, 1024, 2048 | Resynthesize bitstreams (parameter on all D-modules) |
| **Bundle precision CNT_W** | 3, 4, 5, 6 bits | Parameter on `bundle_unit` |
| **Bit-position pruning ratio** | 0%, 50%, 75%, 87.5% | **Runtime only** — load different mask via AXI-Lite, same bitstream |

**Pruning mask:** Per-bit AND before popcount. Bits with low **Fisher discriminability** (from training data) are cleared. Hardware cost is tiny; dynamic energy in AM scales roughly with keep ratio (1 − K/D).

**Expected plot:** Pareto frontier of accuracy vs energy/inference (µJ) — pruning moves you down-left at constant D; shrinking D moves you down-left more aggressively.

### 9.2 Twist 1 — Informed vs random pruning

At each pruning ratio, compare two masks with **identical density** (same number of 1s):

- **Informed:** Fisher discriminability ranking
- **Random:** random bit positions

**Claim to establish:** Informed beats random by ≥ 5 percentage points at iso-budget — proves **bit position matters**, not just bit count. Addresses the "is this just feature selection?" objection.

Python: `make_pruning_masks()` in `hdc_ref.py`.

### 9.3 Twist 2 — Cross-subject mask transfer

Train pruning mask on 18 EMG subjects; evaluate on 18 held-out subjects.

**Either outcome is publishable:**

- Mask generalises → deployable universal pruning
- Mask fails → per-subject calibration required (also a valid finding)

Python: `cross_subject_mask_experiment()` in `hdc_ref.py`.

---

## 10. Experimental plan and metrics

### 10.1 Metrics (from research plan Section 8)

| Metric | How measured |
|---|---|
| Accuracy (%) | Per-subject 5-fold CV (EMG) |
| Throughput (windows/s) | PS cycle counter or stream timestamps |
| End-to-end latency (µs) | Last sample in → class out |
| Energy / inference (mJ or µJ) | Shunt + INA219 on Vcc_int, bare-metal batch |
| Static power (mW) | Same setup, no traffic |
| LUT / FF / BRAM / DSP | Vivado utilisation report |
| f_max (MHz) | Vivado timing report (post-route) |

### 10.2 Required baselines (all four)

1. **ARM-only HDC** — same algorithm in C on Cortex-A9 (with NEON)
2. **AXI-Lite path** — same PL core, register-mapped (shows streaming advantage)
3. **Tiny int8 MLP** — ~5k params, 2 layers
4. **Prior FPGA-HDC literature** — Rahimi 2016, Imani 2017, Schmuck 2019, Hernandez-Cano 2021

### 10.3 Aspirational targets (until measured, label as targets)

- End-to-end latency **< 50 µs**
- **~10×** energy reduction vs ARM-only HDC
- Zero DSP usage — pure logic accelerator

### 10.4 What needs the lab vs VDI

| Task | Remote (VDI) sufficient? |
|---|---|
| Bitstream load, AXI smoke tests, driver iteration | Yes |
| Python Pareto / Twist sweeps | Yes |
| Co-sim regression | Yes |
| Calibrated energy (INA219 + shunt) | Usually needs bench access once |
| Logic analyzer / scope debug | In-person helps if hardware fails |
| Deep advisor meeting on paper framing | In-person valuable, not mandatory |

## 11. Project status: done vs remaining

### 11.1 Completed (May–June 2026 milestone)

- Full RTL datapath: bind, permute, bundle, AM, encoder, core
- AXI4-Lite wrapper + co-sim (200/200 PASS)
- AXI4-Stream wrapper + co-sim with back-pressure (200/200 PASS)
- Python golden reference + EMG baseline frozen at 90.30% @ D=1024
- Per-block and protocol documentation PDFs in `docs/`
- Bare-metal driver stub: `sw/hdc_core_axi_example.c`
- Repository synced to GitHub ( `harsha240yeager/1024-HDC` )

### 11.2 Remaining (hardware + experiments)

1. **Vivado block design** — Zynq PS + AXI-DMA + both wrappers → bitstream
2. **On-board smoke test** — AXI-Lite first, then DMA streaming (VDI OK)
3. **Synthesis sweep** — D and CNT_W variants for Hook A
4. **Measurements** — latency, throughput, energy, area, f_max
5. **Novelty experiments** — Hook A Pareto plots, Twist 1, Twist 2
6. **Baselines** — ARM-only HDC, tiny MLP
7. **Paper / thesis** — figures, write-up, DATE submission prep

**Honest status line:** *RTL and simulation verification are complete; FPGA integration and hardware-characterisation experiments are in progress.*

## 12. Anticipated professor questions

| Question | Prepared answer |
|---|---|
| What is novel vs Rahimi 2016? | They established FPGA HDC on EMG; you add streaming AXI path, three-axis Pareto on real Zynq energy, informed pruning + cross-subject transfer as structured contributions. |
| Why D = 1024? | Matches reproduced baseline; D is a parameter — Pareto sweeps 256–2048. |

| Question | Prepared answer |
|---|---|
| How do you know RTL is correct? | Bit-exact Python golden; seven co-sim harnesses, 500/200 cases each, all PASS; hardest test includes stream back-pressure. |
| What if pruning hurts accuracy? | That is a valid Pareto point — plot the frontier; Twist 1 shows informed > random at same K. |
| Why bare-metal for energy? | No OS background noise; cleanest integration with shunt measurement (per research plan Section 6). |
| What is the bottleneck? | Sequential encode over 20 pairs vs parallel AM popcount — streaming + DMA addresses sustained throughput. |
| Can you finish experiments remotely? | Yes for most bring-up via VDI; calibrated energy may need one bench session. |
| Timeline to submission? | RTL done; Jul–Aug hardware + measurements; Sep 2026 analysis and write-up for DATE 2027. |

## 13. Five-minute meeting structure

| Time | Topic |
|---|---|
| 0:00–0:30 | Problem: edge EMG; HDC on FPGA; DATE-target paper |
| 0:30–1:30 | HDC pipeline: bind → bundle → masked Hamming AM |
| 1:30–3:00 | RTL stack + verification numbers (all co-sims PASS) |
| 3:00–4:00 | AXI-Lite vs Stream; Zynq integration plan |
| 4:00–4:30 | Three contributions: Hook A, Twist 1, Twist 2 |
| 4:30–5:00 | Status, open questions, visit vs remote plan |

## 14. Self-test checklist

Before the meeting, answer these **without notes**:

1. Write the bind equation for one (channel, feature, level) tuple.
2. What does the pruning mask do in hardware?
3. How many co-sim harnesses exist, and what does the stream harness test beyond the core harness?
4. State Hook A, Twist 1, and Twist 2 in one sentence each.
5. Where does training happen vs inference?
6. Why is Python "truth" and not the RTL?
7. What baseline accuracy is frozen, and at what D?

8. Name all four paper baselines.
9. What is 3 beats in, 1 beat out on the stream wrapper?
10. What remains before submission?

If you can answer all ten, you are ready to explain the research end to end.

---

## 15. Key file reference

| Path | Contents |
|---|---|
| `docs/HDC_Research_Plan.html` | Full research plan (print to PDF) |
| `docs/HDC_OnePager.html` | Advisor one-page brief |
| `python_ref/hdc_ref.py` | Bit-exact golden reference |
| `python_ref/run_emg_baseline.py` | Frozen EMG baseline reproduction |
| `rtl/hdc_core_top.sv` | End-to-end inference core |
| `rtl/hdc_stream_wrapper.sv` | AXI4-Stream wrapper |
| `rtl/encoder_top.sv` | EMG window encoder |
| `sim/run_*_cosim.do` | One-command verification harnesses |
| `sw/hdc_core_axi_example.c` | Bare-metal driver stub |
| `README.md` | Repository overview and roadmap |

---

*Document generated for advisor meetings and self-study. Align dates and venue targets with your latest research plan.*